

Programming Assignment 4

Drone Deliveries in a Smart City

Due: 11/9/2025 at 11:59pm

Objective: Students will apply concepts of dynamic programming and implement three solutions using all three techniques including memoization and tabulation.

Assignment Description:

Welcome to Pegasus Express, a cutting-edge delivery company operating in the bustling smart city of Orlando. You have just been assigned a critical task: optimize the delivery routes for a fleet of autonomous drones that transport high-value packages across the city.

Each drone is equipped with a limited battery and can carry only a certain payload. Additionally, battery consumption is no longer linear because heavier packages drain more power per kilometer. Also, these drones start to deteriorate in terms of how much they can hold. This means once a package is equipped to a drone and dropped off to its respective destination, it cannot be reset back its original payload.

Your mission is to develop a set of algorithms that selects the most profitable sequence of deliveries for a single drone, ensuring:

- The drone does not exceed its battery or payload limits.
- The total value of delivered packages is maximized.

This is a high-stakes operation. The future of Pegasus Express's smart delivery system depends on your algorithms. You decided that to verify you retrieved the optimal answer, you applied your dynamic programming concepts from your recent lectures. You decided you are going to implement the recursive solution, recursive top-down solution using memoization, and iterative bottom-up tabulation.

There are a couple of things you can assume for this assignment (to keep things simple) so that you don't have to consider all these hypotheticals in a drone-delivery system scenario:

- The drone can only complete one job at a time. This requires the drone to go to its destination. This also means that the drone is not holding multiple packages at once during its flight.
- The drone does not recharge its battery when coming back to the base station after a job is completed.
- The drone does not make a round trip. Once the drone reaches its destination, the company must retrieve it and bring it back to the base station. Unfortunately, it is not ready for round trips now. This prevents the battery of the drone from draining even further.
- The drone's carrier for package capacity starts to deteriorate after each item it carries. Meaning the drone becomes weaker. This means that after the drone drops off the

package, its capacity is not its original max weight anymore but still the impact of the package's weight. For example, if a drone's capacity is 10 units and it carries an item that weighs 6 units, it will still have a capacity of 4 units after completing the delivery.

- The battery consumption formula per kilometer can be calculated as $\text{batteryCostPerkm} = 1.0 + 0.1 * w$ where w is the weight of the package.

Let's look at a small test case with 6 jobs and the drone has a payload of 4 and battery life of 10:

Job	Distance	Weight	Value
J1	5	2	50
J2	6	3	60
J3	4	4	40
J4	3	2	30
J5	7	5	80
J6	2	1	20

If we calculated the battery drainage for each drone, we would find the following:

$$\text{J1BatteryDrainage} = 1.0 + 0.1 * 2 = 1.2 \quad \text{J1BatteryCost} = 5 * 1.2 = 6$$

$$\text{J2BatteryDrainage} = 1.0 + 0.1 * 3 = 1.3 \quad \text{J2BatteryCost} = 6 * 1.3 = 7.8$$

$$\text{J3BatteryDrainage} = 1.0 + 0.1 * 4 = 1.4 \quad \text{J3BatteryCost} = 4 * 1.4 = 5.6$$

$$\text{J4BatteryDrainage} = 1.0 + 0.1 * 2 = 1.2 \quad \text{J4BatteryCost} = 3 * 1.2 = 3.6$$

$$\text{J5BatteryDrainage} = 1.0 + 0.1 * 5 = 1.5 \quad \text{J5BatteryCost} = 7 * 1.5 = 10.5$$

$$\text{J6BatteryDrainage} = 1.0 + 0.1 * 1 = 1.1 \quad \text{J6BatteryCost} = 2 * 1.1 = 2.2$$

In this scenario the optimal max value we can get is 80 (selecting jobs J1 and J4). The total battery cost is 9.6 and the total payload used is 4.

Note: We could also select J2 and J6 even and still get 80. However, the max value is still 80.

For this assignment, you must follow these **requirements**.

1. Create a class called `DroneDelivery`. In this class, you will implement the three solutions that should generate the same result.
2. You have been provided 5 text files (`jobs1.txt`, `jobs2.txt`, etc...) where each represents a test case that you will test your algorithms with. Each file has the following info on each line that is separated by a whitespace character: `JOB_ID DISTANCE WEIGHT VALUE`
 - a. All files must be in the same directory as your driver and solution files.
 - b. You may assume that the files are properly formatted and doesn't need input validation checking.
 - c. You may assume that the distance in the file is in kilometers.
 - d. You may assume that the weight of the package is in kilograms.

3. You will need to implement the class constructor `DroneDelivery` that takes a string parameter that represents the input text file names. The constructor should help you get your input setup for your respective algorithms.
4. The first method you will implement is a non-static user defined method called `maxCostRecursive`. This method takes two parameters. The first parameter is a double type that represents the maximum battery life of the drone. The second parameter is an integer type that represents the payload capacity of the drone. The method returns an integer value representing the total maximum value that the drone can execute in job deliveries. This method must use top-down recursion. If recursion is not applied in the method, then the grader will automatically give a zero for the method implementation with no partial credit, even if the method returns the correct value. This method must have a worst-case runtime of $O(2^n)$ where n is the number of jobs.
5. The second non-static method will extend to the previous first method. This method is named `maxCostMemo`. Once you successfully implement the first method, you will apply one of the dynamic programming techniques. In the second method you will utilize **memoization**. This method takes two parameters. The parameters are the same as the first method you implemented. The method returns an integer value representing the total maximum value that the drone can execute in job deliveries. This method must use top-down recursion with memoization. If recursion with memoization is not applied in the method, then the grader will automatically give a zero for the method implementation with no partial credit, even if the method returns the correct value. This method must have a worst-case runtime of $O(npb)$ where n is the number of jobs, p is the maximum payload of the drone, and b is the maximum battery life of the drone.
6. The third method will apply the second dynamic programming technique **tabulation**. This method is called `maxCostTab`. This method takes two parameters. The parameters are the same as the first and second methods you implemented. The method returns an integer value representing the total maximum value that the drone can execute in job deliveries. This method must apply the bottom-up tabulation approach with a worst case runtime of $O(npb)$ where n is the number of jobs, p is the maximum payload of the drone, and b is the maximum battery life of the drone. If tabulation is not applied in the method, then the grader will automatically give a zero for the method implementation with no partial credit, even if the method returns the correct value.
7. After you implement all three methods that return the same maximum result, you will also implement a non-static method called `displaySelectedJobs`. This method will construct the solution that led to the maximum value from all three methods. This method should display selected jobs by just displaying the `JOB_ID`. Each one will be displayed on its own line. The method does not return any value.
8. Please make sure that you have comments in your methods. If there aren't any comments in your file, the graders will deduct points.
9. You may create additional helper methods and attributes if needed as long as they are implemented and called in your solution file and NOT called from the driver file. Adding extra methods to call in the driver file will not match to what the graders will use evaluate

your code. This will result in a low score and students will not be able to ask for these points back.

10. You may create additional helper classes as long as those classes are defined in your solution file and not the driver file. If you define it in your driver file, then your program will not compile properly for the graders and they will automatically give no points for the majority of the rubric except for code style, comments, and code submission. Students will not be able to ask for these points back.
11. If your code does not compile or is stuck in an infinite loop, you will receive no credit on any of the rubric categories except for code style/comments, assignment directions, and code submission/header.

A driver file (`DroneDeliveryDriver.java`) has been provided for you to show you how the methods are called along with 5 test cases. You were also provided a test script that the graders will use in evaluating your code. It is your responsibility to run the script with your code before submitting to avoid any issues with the grading process.

What to submit: Submit a file called `DroneDelivery.java` to webcourses. You are not required to submit the driver file as that will be provided for the graders to test your code. Please make sure the driver file provided works for your code. Any name changes may cause your program not to work when graded, which will result in a lower score on the assignment and would not be changed.

Comment Header: Please make sure to provide the appropriate comment header (like in the Eustis assignment) as the first thing on the top of your file. Otherwise, if your comment header is done incorrectly including not being placed literally at the top of the file will result point deductions in accordance with the rubric. They must be formatted as such and placed on the first line of the file with no whitespace above:

```
/* INSERT YOUR NAME HERE
   Dr. Steinberg
   COP3503 Fall 2025
   Programming Assignment 4
*/
```

What happens if my code doesn't compile? How will my code be graded? If your code falls under not compiling, you will not be eligible to receive any points on the test cases, method implementations (all three versions), categories. That means you would receive 0 points in those respective categories. The only full points you would be eligible for are code style, comments, and code submission/header if directions were followed. You will also receive no marks if your code does not compile.

What happens if my code compiles, but crashes or throws an exception or is stuck in an infinite loop? How will my code be graded? If your code compiles, but crashes or throws an exception or is stuck in an infinite loop, you will not be eligible to receive any points on the test cases, method implementations (all three versions), categories. That means you would receive 0 points in those respective categories. The only full points you would be eligible for are code style, comments, and code submission/header if directions were followed. You will also receive low marks if your code does compile.

Important Note for running Eustis: Many of you are probably using IDEs like Netbeans and Eclipse to build your Java Solutions. Please note that some of these IDEs will automatically place your Java file in some sort of package. Please make sure your Java file is not defined in some package as this can result package private errors. Any such error that occurs during the grading will not be fixed and points will be deducted as such in accordance with the respective categories in the rubric. Also, DO NOT create a main method in your solution file!! This will result in your code not running properly with the driver file which will result in points being deducted from the respective categories.

Code Style Requirements: It is expected that students will follow the provided code style guidelines. These guidelines are available in webcourses on the assignment page. Students who violate any of the guidelines will receive point deductions!

AI Syllabus Policy Reminder

The following comes from our course syllabus.

Use of Generative Artificial Intelligence (GenAI) is prohibited. Use of GenAI tools via website, app, or any other access, is not permitted in this class. All components of assignments in this course must be independently and originally completed by the student. Representing work created by GenAI as your own will be treated as plagiarism.